FedHydra: recap

Reviewed

Parser group

```
parser.add_argument('-g', '--groups', type=int, default=64,
help='Choose the number of HYDRA groups to use')
```

Aggiunta la possibilità di specificare il numero di gruppi da creare. Default 64

Group

```
top_k_per_client = (n_groups * 1) // n_clients
```

```
seeds = np.arange(n_clients * n_groups).reshape(n_clients,
n_groups)
```

In HYDRA il numero di kernel per gruppo è una costante strutturale fissata a $k = 8$ mentre la dimensione del modello è controllata dal numero di gruppi g, passato da riga di comando tramite il parametro --groups (n_groups).

Sostituendo n_kernels con n_groups in np.arange(n_clients * n_groups).reshape(n_clients, n_groups), il sistema assegna inizialmente a ciascuno degli N client esattamente n_groups seed interi univoci e non sovrapposti. Ogni seed istanzia un HydraGroup completo.

Allo stesso modo, la regola di riduzione dimensionale di Fed2RC è traslata sui gruppi ponendo top_k_per_client = n_groups // n_clients. In questo modo, ad ogni round, ciascun client seleziona e propone al server una frazione $\frac{1}{N}$ dei gruppi migliori.

Pesi

```
# Estrazione dei pesi aggregata su tutte le classi (per ges
tire sia binari che multiclasse)
w = np.sum(np.abs(model.coef_), axis=0) if model.coef_.ndim
> 1 else np.abs(model.coef_)

num_seeds = len(client_seeds)
features_per_group = x_trans.shape[1] // num_seeds

# Calcolo del punteggio di ogni singolo HydraGroup (Norma L
1 del blocco)
seed_scores = np.zeros(num_seeds)
for i in range(num_seeds):
    start_idx = i * features_per_group
    end_idx = start_idx + features_per_group
    group_weights = w[start_idx:end_idx]
    seed_scores[i] = np.sum(np.abs(group_weights))

top_k_idx = np.argsort(-seed_scores)[:top_k_per_client]
top_k_seeds = client_seeds[top_k_idx]
```

In HYDRA un singolo seed (un HydraGroup) genera un blocco di feature il vettore dei pesi w del RidgeClassifierCV locale ha dimensione num_seeds * M. Fare np.argsort direttamente sui singoli pesi mi ha causato inizialmente un IndexError.

Per risolvere questo problema, ho partizionato il vettore dei pesi in blocchi contigui di dimensione features_per_group e ho calcolato l'importanza complessiva del gruppo tramite la norma dei pesi del blocco (np.sum(np.abs(group_weights))), sommando i moduli lungo axis=0 per tenere conto di tutte le classi nei problemi multiclasse come uno dei dataset di esempio.

```
A_global = None
b_global = None
A_locals, b_locals = [], []
for client_id in range(n_clients):
    ...
    A_local = x_trans.T @ x_trans
```

```
    b_local = x_trans.T @ y_onehot

    if A_global is None:
        n_features_reali = A_local.shape[0]
        A_global = np.zeros((n_features_reali, n_features_r
eali))
        b_global = np.zeros((n_features_reali, n_classes))

    A_locals.append(A_local)
    b_locals.append(b_local)
    A_global += A_local
    b_global += b_local
```

In Fed2RC, le matrici globali venivano allocate prima dell'uso con dimensione (final_num_kernels, final_num_kernels) perché il numero di feature coincideva col numero di seed rimasti. Con HYDRA, invece, ho risolto Impostando A_global = None e istanziando le matrici globali alla prima iterazione leggendo n_features_reali = A_local.shape[0].

HydraGroup in Utils.py

```
class HydraGroup:
def init(self, seed, ts_length):
self.hydra = HydraTransformer(n_kernels = 8, n_groups = 1,
random_state = int(seed))
self.hydra.fit(np.zeros((1, 1, ts_length)))

def transform(self, X):
    if len(X.shape) <= 1:
        X = X.reshape(1, -1)
    X_transformed = self.hydra.transform(np.expand_dims(X,
1))
    return X_transformed
```

```
def hydra_transform_seeds(X, seeds, ts_length):
K = [HydraGroup(int(seed), ts_length) for seed in seeds]
```

```
return np.concatenate([k.transform(X) for k in K], axis=1)
```

In Fed2RC, la classe RocketKernel fissava num_kernels=1 e ppv_only=True, garantendo una corrispondenza 1:1 tra un seed intero e una singola feature. In HYDRA, invece, i kernel competono tra loro. Non essendo possibile separare un kernel dagli altri avversari del suo gruppo senza alterare i conteggi, ho ridefinito l'unità atomica associata a ciascun seed creando la classe HydraGroup. Fissando n_groups = 1 e n_kernels = 8, ogni seed inizializza una sfida di 8 kernel su tutte le dilatazioni della serie. La funzione hydra_transform_seeds istanzia i gruppi associati ai seed del client e concatena orizzontalmente i blocchi di feature generati da ciascun gruppo.

Considerazioni sull’esecuzioni

L’esecuzione di FedHydra a differenza di Fed2RC ci ha messo almeno 1 ora sempre sugli stessi dataset (4). Sicuramente una buona parte del tempo è stata dovuta alla presenza di 4 Experiment Seeds. Fed2Rc era stato eseguito con solo 1 singolo seed.